

Project summary: 2D Lid-Driven Cavity CFD Solver in C++ with OpenMP

Author: Tyler Warner

I built a working introductory computational fluid dynamics solver from scratch in C++, parallelized portions of it with OpenMP, visualized the resulting flow field with Python, and compared the solution against a well-known CFD benchmark.

I am solving a 2D lid-driven cavity at a Reynolds number of 100. It's a classic CFD validation problem because the geometry is extremely simple, while the resulting fluid motion exercises many of the fundamental ideas behind incompressible CFD.

1. The Physical Problem

I created a square cavity:

$$L_x = L_y = 1$$

filled with an incompressible fluid. Three walls are stationary, while the top wall moves to the right at

$$U_{lid} = 1$$

My code uses

$$\rho = 1, \nu = 0.01$$

giving

$$Re = \frac{U_{lid}L}{\nu} = \frac{(1)(1)}{(0.01)}.$$

The moving wall transfers momentum into the fluid. Because the other three walls are closed, the fluid can't simply travel to the right indefinitely. Instead, it circulates and forms a large clockwise primary vortex.

That's exactly what your final streamline and vector plots now show.

2. What Equations My Program is Approximately Solving

The underlying physics are the 2D incompressible Navier-Stokes equations.

For horizontal velocity u :

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial x} + \nu \nabla^2 u$$

and vertical velocity v :

$$\frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial y} + \nu \nabla^2 v.$$

The incompressibility constraint is:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

This last equation is extremely important. It says that for an incompressible fluid, the velocity field must be divergence-free—essentially, fluid can't spontaneously appear or disappear inside a grid cell.

3. Turning Continuous Physics into Something a Computer Can Solve

I discretized the 1 x 1 cavity into a 101 x 101 grid. Therefore:

$$\Delta x = \Delta y = \frac{1}{100} = 0.01.$$

I then properly named all needed variables in 2D C++ arrays and implemented them as `vector<vector<double>>`.

Instead of calculating derivatives analytically, I approximated them from neighboring grid points.

For example:

$$\frac{\partial u}{\partial x} \approx \frac{u_{i+1,j} - u_{i-1,j}}{\Delta x}.$$

That's the central-difference approximation implemented into my predictor. Using one of the fundamental ideas behind CFD where you turn differential equations into algebraic equations on a grid.

4. Predictor-Corrector Approach

My program doesn't solve u , v , and p simultaneously. Instead, it essentially uses a projection/fractional-step approach. First, it predicts intermediate velocities:

$$u^*, v^*$$

without applying the new pressure field. Conceptually:

$$\text{old velocity} \xrightarrow{\text{convection+diffusion}} \text{predicted velocity}$$

My predictor includes both convective transport

and viscous diffusion

$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} \quad v \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right).$$

I implemented these calculations in the main parallel predictor loop.

5. Pressure then Enforces Incompressibility

The predicted velocity isn't necessarily divergence-free. So, I calculate its divergence:

$$\frac{\partial u^*}{\partial x} + \frac{\partial v^*}{\partial y}$$

and use that to construct the right-hand side of a pressure Poisson equation. My code does this immediately after applying the intermediate velocity boundary conditions.

I then iteratively solve for pressure. This is where I learned a valuable lesson of CFD. My results did not match what I was expecting, it taught me that successful compilation and attractive plots do not mean a numerical simulation is correct.

6. Pressure Correction

Once pressure has been calculated, you correct the predicted velocities according to the pressure gradient:

$$u^{n+1} = u^* - \frac{\Delta t}{\rho} \frac{\partial p}{\partial x}$$

and

$$v^{n+1} = v^* - \frac{\Delta t}{\rho} \frac{\partial p}{\partial y}.$$

The cycle is therefore approximately: Current $u, v \rightarrow$ convection + diffusion $\rightarrow$ predicted $u^*, v^* \rightarrow$ calculate divergence $\rightarrow$ pressure Poisson equation $\rightarrow$ pressure $p \rightarrow$ correct velocities $\rightarrow$ new $u, v \rightarrow$ repeat.

7. Boundary Conditions

The cavity walls have no-slip velocity conditions.

Bottom:

$$u = v = 0$$

Left/right:

$$u = v = 0$$

Top:

$$u = U_{lid} = 1, \quad v = 0$$

My program explicitly reapplies these conditions after pressure correction. This is why the top of my velocity-magnitude plot approaches:

$$|\mathbf{V}| \approx 1.$$

The lid is physically driving the rest of the simulation.

8. Parallel Programming with OpenMP

I implemented parallel computing to improve the speed and efficiency of my program. For computationally expensive nested grid loops, I'm using:

```
#pragma omp parallel for collapse(2)
```

For example, the predictor calculation is parallelized this way. Instead of one CPU thread processing roughly 10,000 interior grid points sequentially, OpenMP can distribute the work among multiple CPU threads.

9. Convergence Monitoring

Rather than blindly running a fixed number of iterations, I've started monitoring whether the solution is changing. My code computes this using an OpenMP max reduction, velocity residual measures approximately:

$$R_u = \max(|u^{n+1} - u^n|, |v^{n+1} - v^n|).$$

I also calculate a continuity error based on:

$$\left| \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right|,$$

which is directly connected to incompressible mass conservation.

Lastly, I have also implemented a pressure-equation residual as another numerical diagnostic. The most recent simulation reached the velocity stopping criterion at approximately iteration 9142, with a tolerance of 10^{-6} .

10. Post-Processing with Python

My C++ based program outputs the variables into a .csv file at the end of my solver. I then built a separate Python post-processing program using NumPy, pandas, and Matplotlib. Producing plots of velocity vectors, streamlines, velocity-magnitude contours, and pressure contours. C++ performs computational work while Python handles visualization and analysis.

11. Validation Against Ghia et al.

Rather than deciding “My picture looks like CFD, therefore it’s correct.” I compared my solution against established lid-driven-cavity benchmark data at:

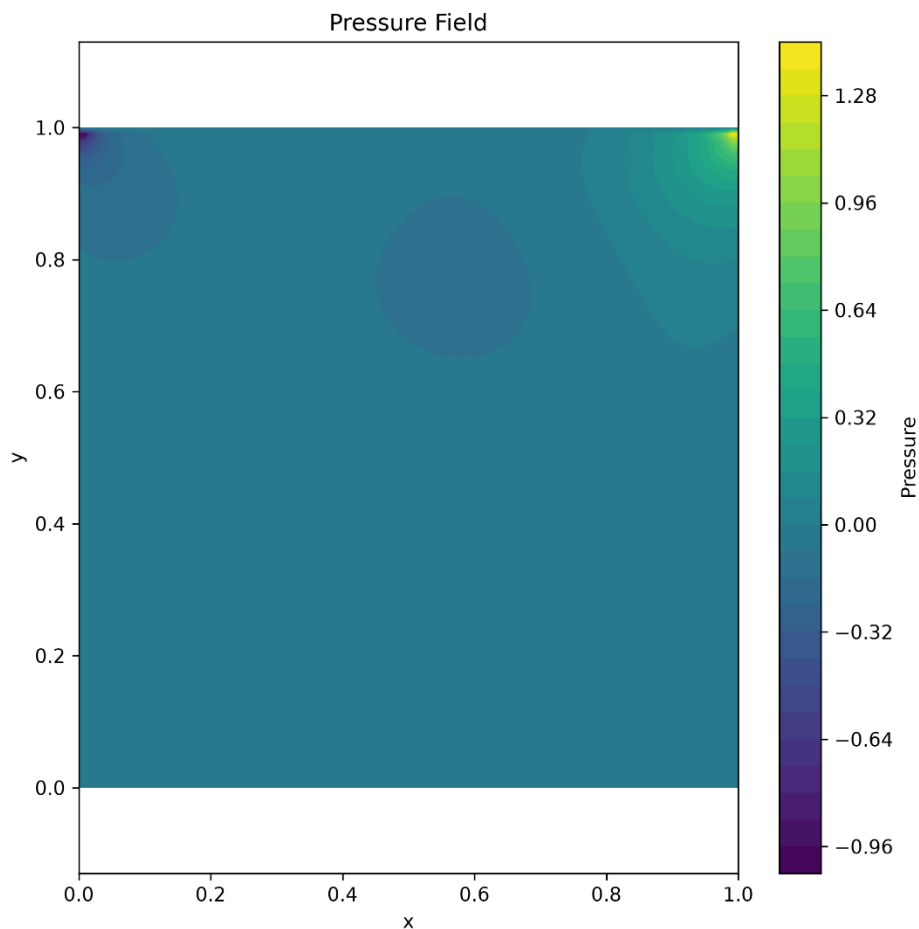
$$Re = 100.$$

I extracted two profiles:

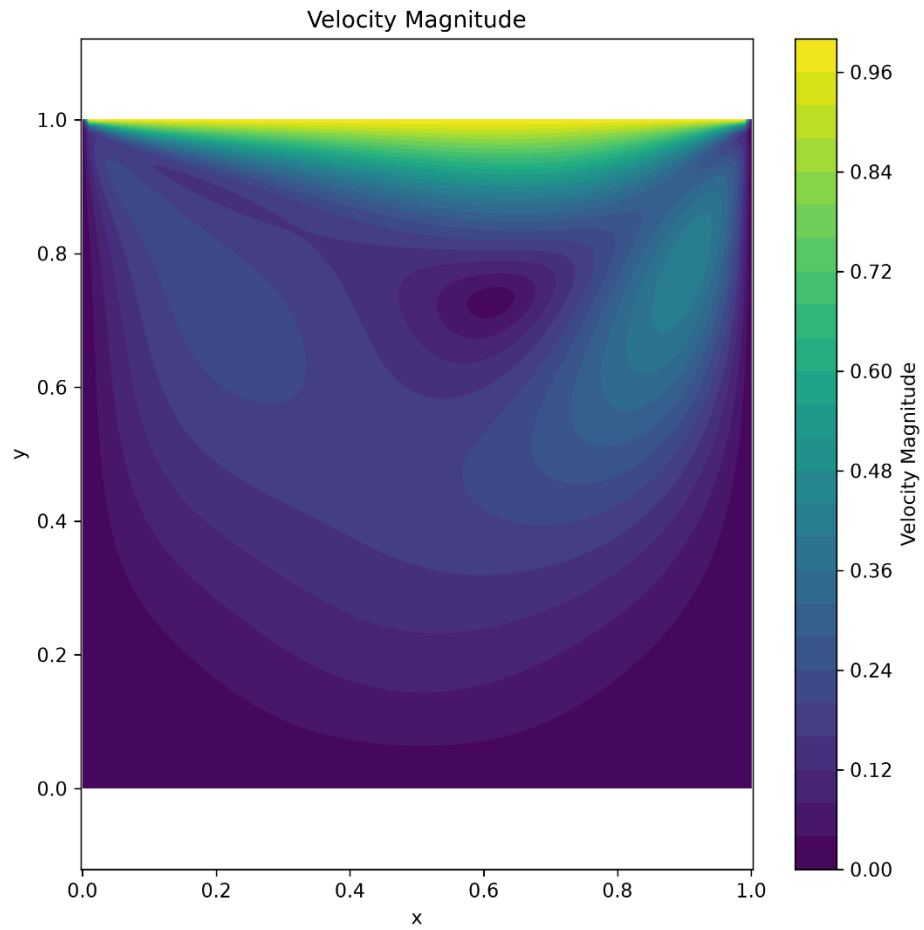
$$u(0.5, y) \text{ and } v(x, 0.5).$$

My Python validation plots then put my C++ results against the benchmark points. My curves follow the benchmark reasonable closely, as seen in the results below. That gives actual evidence and validity that my solver is reproducing the expected solution—not just producing a vortex-shaped picture.

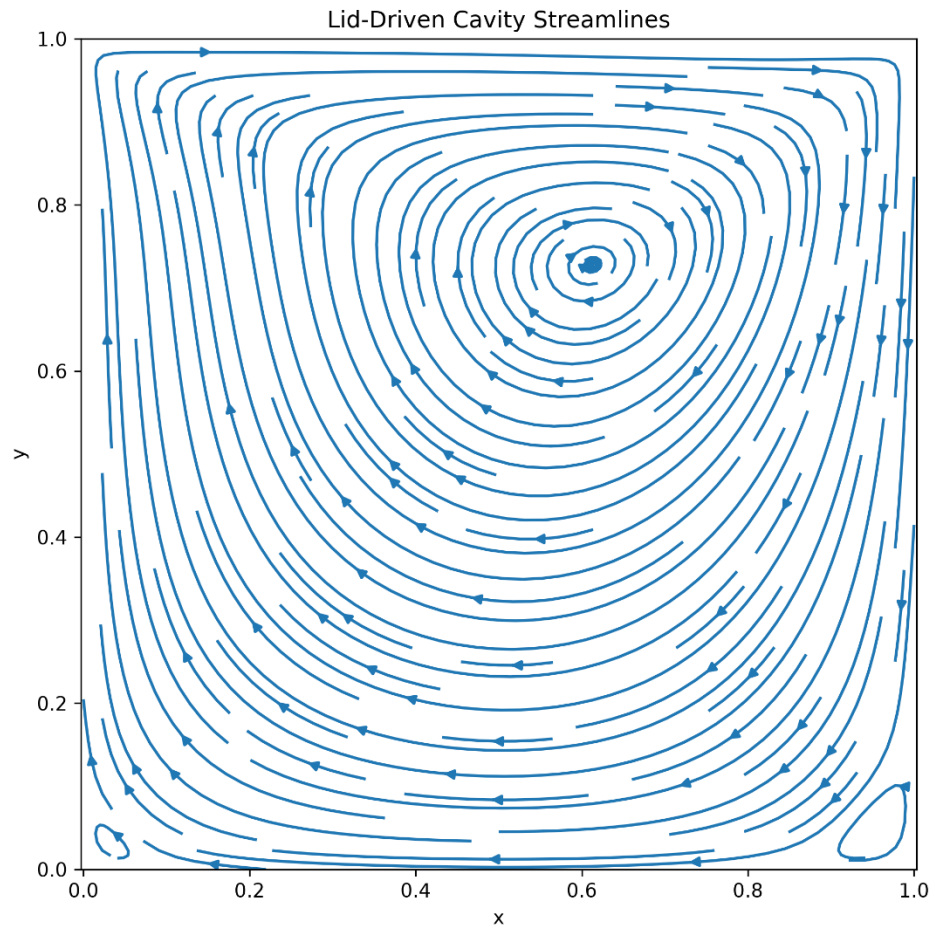
12. Results



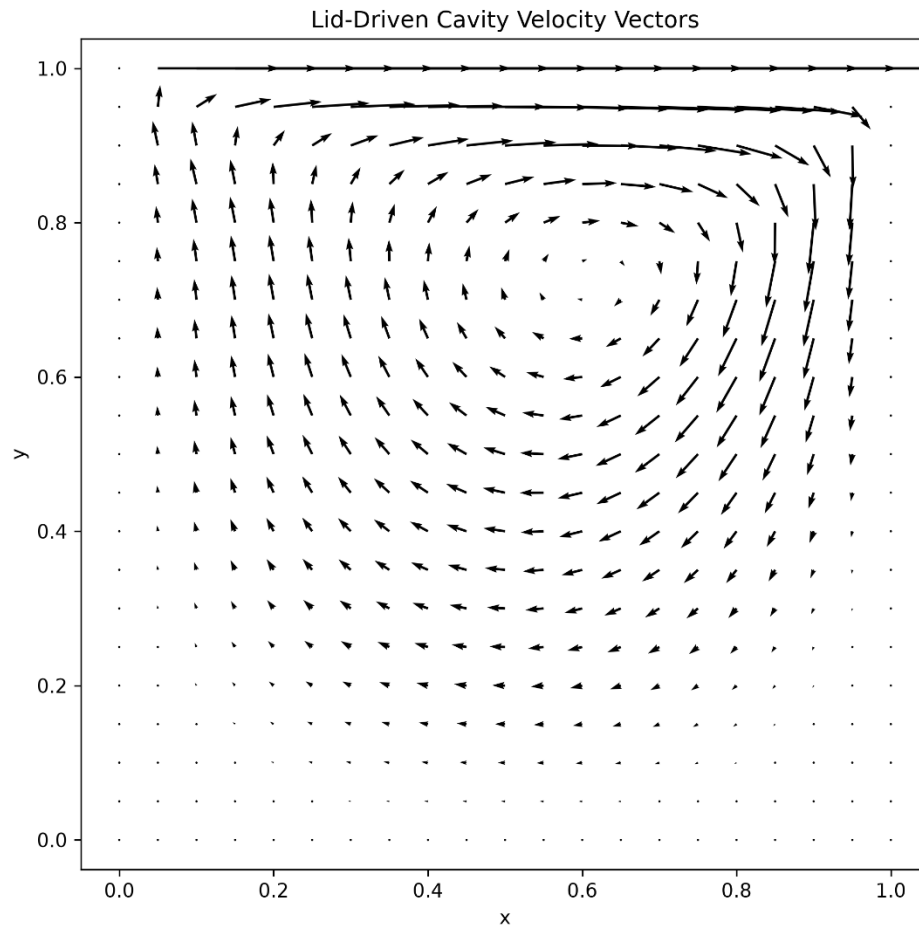
Pressure Field—Shows the pressure distribution throughout the cavity, including the strongest pressure variations near the upper corners where the moving lid interacts with the stationary walls.



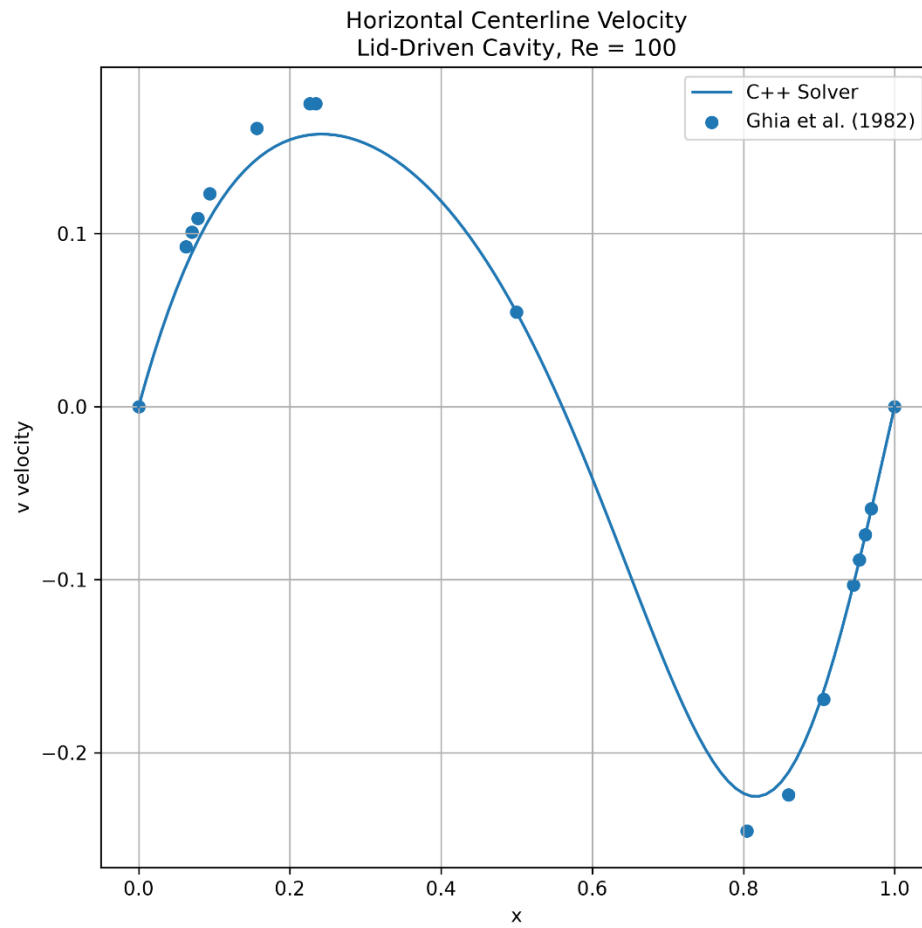
Velocity Magnitude Contour—Shows the magnitude of the fluid velocity throughout the cavity, with the highest velocities occurring along the moving upper lid.



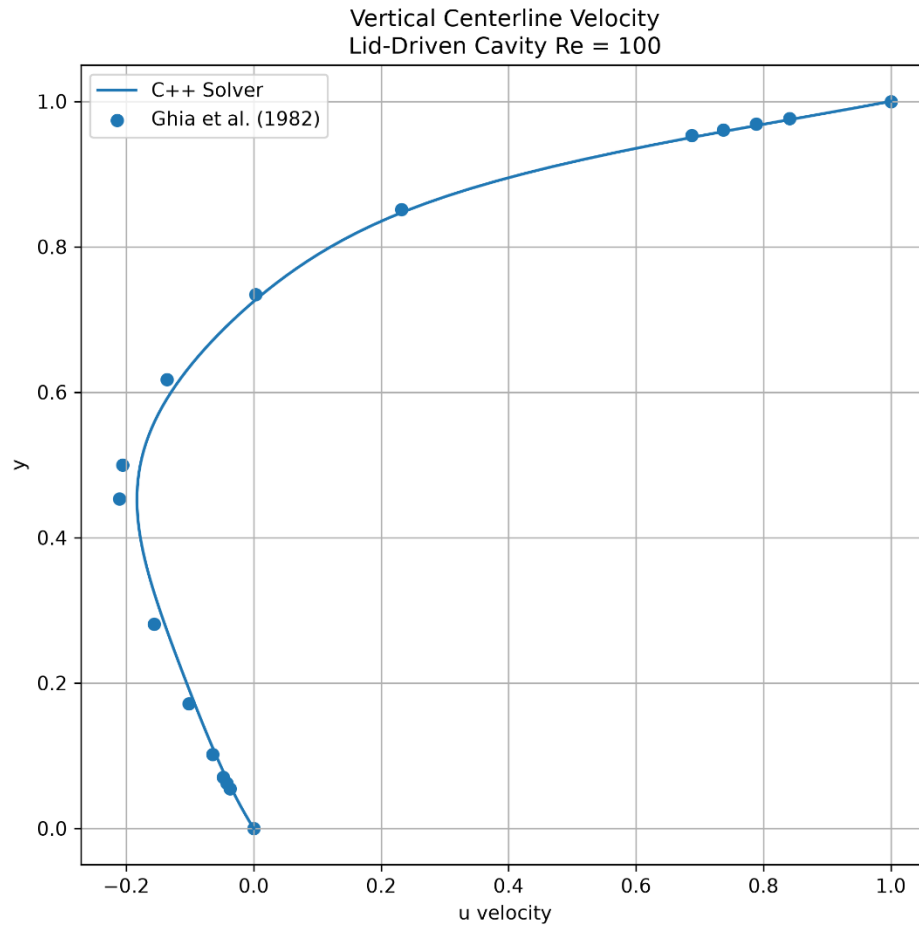
Flow Streamlines—Visualizes the overall flow structure, showing the dominant clockwise recirculation vortex and smaller secondary vortices near the lower corners.



Velocity Vector Field—Displays the direction and relative magnitude of the velocity field, clearly illustrating the clockwise circulation generated by the moving lid.



Horizontal Centerline Velocity Validation—Compares the calculated vertical velocity v along the cavity's horizontal centerline with the Ghia et al. (1982) benchmark data at $Re = 100$.



Vertical Centerline Velocity Validation—Compares the calculated horizontal velocity u along the cavity's vertical centerline with the Ghia et al. (1982) benchmark data at $Re = 100$.